

60. URL objects

Built-in `URL` class URLs banane aur parse karne ka aasan interface deti hai. Networking methods ko sach me `URL` object nahi chahiye (strings kaafi hain), par kabhi kabhi bahut kaam aata hai.

URL banana

```
js  
  
new URL(url, [base])
```

- `url`: poora URL, ya sirf path (agar `base` diya ho)
- `base`: optional base URL, `url` sirf path ho to usse relative banta hai

```
js  
  
let url1 = new URL('https://javascript.info/profile/admin');  
let url2 = new URL('/profile/admin', 'https://javascript.info'); // same URL  
  
let url = new URL('https://javascript.info/profile/admin');  
let newUrl = new URL('tester', url); // https://javascript.info/profile/tester
```

Parse karke components milte hain:

```
js  
  
let url = new URL('https://javascript.info/url');  
url.protocol; // "https:"  
url.host; // "javascript.info"  
url.pathname; // "/url"
```

Components:

- `href`: poora URL (`url.toString()` jaisa)
- `protocol` (`:` par khatam)
- `host`, `pathname`
- `search`: parameters ki string, `?` se shuru
- `hash`: `#` se shuru
- `user`, `password`: HTTP authentication ho to (rare)

`fetch`, `XMLHttpRequest` me string ki jagah `URL` object seedha de sakte ho (string me convert ho jaata hai).

`searchParams` ("?...")

Search params me spaces, non-latin letters ho sakte hain jinhe **encode** karna padta hai.

`url.searchParams` (`URLSearchParams` type) iske liye methods deta hai:

- `append(name, value)`
- `delete(name)`
- `get(name)`
- `getAll(name)` (same naam ke kai, jaise `?user=John&user=Pete`)
- `has(name)`
- `set(name, value)` (set/replace)
- `sort()` (kam use)
- `Map` ki tarah iterable

js

```
let url = new URL('https://google.com/search');
url.searchParams.set('q', 'test me!');
alert(url); // https://google.com/search?q=test+me%21

url.searchParams.set('tbs', 'qdr:y');
alert(url); // https://google.com/search?q=test+me%21&tbs=qdr%3Ay

for (let [name, value] of url.searchParams) {
  alert(`${name}=${value}`); // decoded: q=test me!, tbs=qdr:y
}
```

Parameters **automatically encode** hote hain.

Encoding

Standard **RFC3986** batata hai URL me kaun se characters allowed hain. Baaki (non-latin letters, spaces) UTF-8 codes me, `%` ke saath encode hote hain, jaise `%20` (space ko `+` bhi likh sakte hain, purani wajah se).

`URL` objects ye sab khud karte hain, bas unencoded values do:

js

```
let url = new URL('https://ru.wikipedia.org/wiki/Тест');
url.searchParams.set('key', 'Ъ');
alert(url); // path aur parameter dono encode
```

(Cyrillic letter UTF-8 me 2 bytes ka hota hai, isliye har letter ke liye 2 (%..).)

Strings ka encoding

`URL` objects se pehle strings hi use hoti thi. Ab bhi kar sakte ho (kabhi code chhota hota hai), par khud encode/decode karna padta hai:

- `encodeURIComponent` / `decodeURIComponent`: **poore URL** ke liye
- `encodeURIComponent` / `decodeURIComponent`: **URL ke ek component** (search parameter, hash, pathname) ke liye

Farak: URL me `:`, `?`, `=`, `&`, `#` allowed hain (URL ka structure). Par kisi ek component ke andar ye characters encode hone chahiye, warna structure toot jaata hai.

- `encodeURIComponent` sirf **poori tarah forbidden** characters encode karta hai.
- `encodeURIComponent` unke saath `#`, `$`, `&`, `+`, `,`, `/`, `:`, `;`, `=`, `?`, `@` bhi encode karta hai.

js

```
encodeURIComponent('http://site.com/привет'); // poore URL ke liye

let music = encodeURIComponent('Rock&Roll');
`https://google.com/search?q=${music}`; // ...q=Rock%26Roll (sahi)

let music2 = encodeURIComponent('Rock&Roll');
`https://google.com/search?q=${music2}`; // ...q=Rock&Roll (galat! q=Rock aur
```

To har search parameter ke liye `encodeURIComponent` use karo (naam aur value dono encode karna safest).

`URL` vs `encode*` ka farak

`URL` / `URLSearchParams` naye **RFC3986** par, `encode*` functions purane **RFC2396** par based hain. Kuch differences, jaise IPv6 addresses:

js

```
let url = 'http://[2607:f8b0:4005:802::1007]/';
encodeURIComponent(url); // http://%5B2607:f8b0:4005:802::1007%5D/ (galat, brackets
new URL(url); // http://[2607:f8b0:4005:802::1007]/ (sahi)
```

Aise cases rare hain, `encode*` zyaaadatar theek chalte hain.

Summary

- `new URL(url, [base])` se URL banao/parse karo.
- Parameters ke liye `url.searchParams` (auto-encode).
- Strings ke saath: poore URL ke liye `encodeURIComponent`, parameters/components ke liye `encodeURIComponent`.